HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN

BÁO CÁO BÀI THỰC HÀNH
HỌC PHẦN: THỰC TẬP CƠ SỞ
MÃ HỌC PHẦN: INT13147

BÀI THỰC HÀNH 4.2
LẬP TRÌNH THUẬT TOÁN MẬT MÃ HỌC

Sinh viên thực hiện:

B22DCAT303 - Nguyễn Khắc Trí

Giảng viên hướng dẫn: PGS. TS. Hoàng Xuân Dậu

HỌC KỲ 2 NĂM HỌC 2024-2025

MỤC LỤC

DANH MỤC CÁC HÌNH VẼ 3
DANH MỤC CÁC TỪ VIẾT TẮT 4
CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ BÀI THỰC HÀNH 5
1.1 Mục đích 5
1.2 Tìm hiểu lý thuyết 5
1.2.1 Tìm hiểu về lập trình số lớn với các phép toán cơ bản 5
1.2.2 Tìm hiểu về giải thuật mật mã khóa công khai RSA 6
CHƯƠNG 2. NỘI DUNG THỰC HÀNH 9
2.1 Chuẩn bị môi trường 9
2.2 Các bước thực hiện 9
2.2.1 Lập trình thư viện số lớn với các phép toán cơ bản, thử nghiệm chứng minh thư viện hoạt động tốt với các ví dụ phép toán cho số lớn 9
2.2.2 Lập trình giải thuật mã hóa và giải mã 11
2.2.3 Thử nghiệm mã hóa và giải mã chuỗi ký tự: “I am B22DCAT303” 11
TÀI LIỆU THAM KHẢO 13

DANH MỤC CÁC HÌNH VẼ

Hình 1 Hàm tính ước chung lớn nhất 9
Hình 2 Hàm tính a^b mod m 9
Hình 3 Hàm tìm số nghịch đảo d sao cho d.e mod m = 1 10
Hình 4 Hàm sinh khóa, để đơn giản ta chọn e = 65537 10
Hình 5 Hàm mã hóa và giải mã 11
Hình 6 Thử nghiệm code với mã hóa và giải mã 11
Hình 7 Kết quả chạy chương trình 12

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh/Giải thích	Thuật ngữ tiếng Việt/Giải thích
RSA	Rivest–Shamir–Adleman	Thuật toán mã hóa khóa công khai RSA
GCD	Greatest Common Divisor	Ước chung lớn nhất
AES	Advanced Encryption Standard	Chuẩn mã hóa nâng cao
SSH	Secure Shell	Giao thức kết nối an toàn
SSL/TLS	Secure Sockets Layer / Transport Layer Security	Lớp bảo mật socket / Bảo mật tầng truyền tải
HTTPS	Hypertext Transfer Protocol Secure	Giao thức truyền siêu văn bản bảo mật
CLI	Command Line Interface	Giao diện dòng lệnh
IDE	Integrated Development Environment	Môi trường phát triển tích hợp

CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ BÀI THỰC HÀNH

1.1 Mục đích

- Sinh viên tìm hiểu một giải thuật mã hóa phổ biến và lập trình được chương trình mã hóa và giải mã sử dụng ngôn ngữ lập trình phổ biến như C/C++/Python/Java, đáp ứng chạy được với số lớn.

1.2 Tìm hiểu lý thuyết

1.2.1 Tìm hiểu về lập trình số lớn với các phép toán cơ bản

1.2.1.1 Một số lý thuyết cơ bản về lập trình số nguyên lớn:

Biểu diễn số nguyên lớn: Trong lập trình số nguyên lớn, số này thường được biểu diễn dưới dạng mảng hoặc danh sách các chữ số. Mỗi phần tử trong mảng hoặc danh sách này thường lưu trữ một chữ số của số nguyên.

- Cộng và Trừ:
  - Các phép toán cộng và trừ có thể được thực hiện bằng cách duyệt qua từng chữ số của hai số và thực hiện phép toán tương ứng từng cặp chữ số, kèm theo việc xử lý nhớ và mượn nếu cần.
  - Bắt đầu từ chữ số đơn vị và tiếp tục lần lượt đến chữ số hàng đơn vị, thực hiện phép toán cộng hoặc trừ tương ứng.
- Nhân:
  - Phép nhân hai số nguyên lớn có thể thực hiện bằng thuật toán nhân dài (long multiplication) hoặc các thuật toán nhân nhanh hơn như nhân Karatsuba hoặc nhân Toom-Cook.
  - Thuật toán nhân dài thực hiện phép nhân tương tự như cách thực hiện phép nhân trong toán học cơ bản: duyệt qua từng chữ số của một số và nhân với từng chữ số của số kia, sau đó cộng các kết quả lại.
- Chia:
  - Phép chia hai số nguyên lớn có thể thực hiện bằng thuật toán chia dài (long division) hoặc các thuật toán chia hiệu quả hơn như thuật toán chia như thuật toán chia nhân bán (divide-and-conquer division) hoặc thuật toán chia nhưng tốt hơn (Newton-Raphson division).
  - Tương tự như phép nhân, thuật toán chia dài duyệt qua từng chữ số của số chia và thực hiện phép chia tương ứng.
- Tính Lũy Thừa:
  - Phép tính lũy thừa của một số nguyên lớn có thể được thực hiện bằng thuật toán lũy thừa nhanh (exponentiation by squaring). Thuật toán này giảm thiểu số lượng phép toán bằng cách phân tích số mũ thành các phần nhỏ và tính lũy thừa của từng phần.

- Tìm Ước Chung Lớn Nhất (GCD):
  - Phép tìm ước chung lớn nhất của hai số nguyên lớn có thể được thực hiện bằng thuật toán Euclid mở rộng (extended Euclidean algorithm) hoặc thuật toán nhị phân (binary GCD algorithm). Cả hai thuật toán này đều dựa trên việc lặp đi lặp lại một số phép chia và tính toán dựa trên phần dư.

Trong lập trình số nguyên lớn, việc tối ưu hóa và cải thiện hiệu suất là rất quan trọng. Điều này bao gồm việc sử dụng các thuật toán hiệu quả nhất để thực hiện các phép toán cơ bản và giải quyết các vấn đề liên quan đến hiệu năng và bộ nhớ.

Trong Python, số nguyên không bị giới hạn về giá trị. Tuy nhiên, số nguyên có thể lớn đến mức nào phụ thuộc vào bộ nhớ có sẵn trên hệ thống mà bạn đang sử dụng. Chính vì vậy, ta có thể xử lý số nguyên lớn dễ dàng với Python

1.2.2 Tìm hiểu về giải thuật mật mã khóa công khai RSA

1.2.2.1 Giải thuật mã hóa khóa công khai

Mã hóa khóa bất đối xứng (Hay còn gọi là mã hóa khóa công khai): là phương pháp mã hóa mà khóa sử dụng để mã hóa sẽ khác với khóa dùng để giải mã. Khóa dùng để mã hóa gọi là khóa công khai và khóa dùng để giải mã gọi là khóa bí mật. Tất cả mọi người đều có thể biết được khóa công khai (kể cả hacker), và có thể dùng khóa công khai để mã hóa thông tin. Nhưng chỉ có người nhận mới nắm giữ khóa bí mật, nên chỉ có người nhận mới có thể giải mã được thông tin.

Đặc điểm nổi bật của các hệ mã hóa khóa bất đối xứng là kích thước khóa lớn, lên đến hàng ngàn bit. Do vậy, các hệ mã hóa dạng này thường có tốc độ thực thi chậm hơn nhiều lần so với các hệ mã hóa khóa đối xứng với độ an toàn tương đương. Mặc dù vậy, các hệ mã hóa khóa bất đối xứng có khả năng đạt độ an toàn cao và ưu điểm nổi bật nhất là việc quản lý và phân phối khóa đơn giản hơn do khóa công khai có thể phân phối rộng rãi.

Hệ thống mật mã hóa khóa công khai có thể sử dụng với các mục đích:

- Mã hóa: giữ bí mật thông tin và chỉ có người có khóa bí mật mới giải mã được.
- Tạo chữ ký số: cho phép kiểm tra văn bản có phải đã được tạo với một khóa bí mật nào đó hay không.
- Thỏa thuận khóa: cho phép thiết lập khóa dung để trao đổi thông tin giữa hai bên.

Các giải thuật mã hóa khóa bất đối xứng điển hình bao gồm: RSA, Rabin, ElGamal, McEliece và Knapsack.

1.2.2.2 Giải thuật mã hóa khóa công khai RSA

Thuật toán mã hóa RSA là một thuật toán mã hóa khóa công khai. Thuật toán RSA được xây dựng bởi các tác giả Ron Rivest, Adi Shamir và Len Adleman tại học viện MIT vào năm 1977, và ngày nay đang được sử dụng rộng rãi. Về mặt tổng quát thuật toán RSA là một phương pháp mã hóa theo khối. Trong đó thông điệp M và bản mã C là các số nguyên từ 0 đến 2i với i số bit của khối. Kích thước thường dùng của i là 1024 bit. Thuật toán RSA

sử dụng hàm một chiều là vấn đề phân tích một số thành thừa số nguyên tố và bài toán Logarith rời rạc.

Thuật toán

Để thực hiện mã hóa và giải mã, thuật toán RSA dùng phép lũy thừa modulo của lý thuyết số.

Quá trình sinh khóa

- Chọn hai số nguyên tố lớn p và q và tính N = pq. Cần chọn p và q sao cho: M < $2i-1$ < N < $2i$ với i là chiều dài bản rõ.
- Tính Φ(n) = (p - 1)(q - 1)
- Tìm một số e sao cho: {e và Φ(n) là 2 số nguyên tố cùng nhau và 0 < e < Φ(n)}
- Tìm một số d sao cho: e.d ≡ 1 mod Φ(n) (hay: d = $e-1$ mod Φ(n)) (d là nghịch đảo của e trong phép modulo Φ(n))
- Chọn khóa công khai Ku là cặp (e, N), khóa riêng Kr là cặp (d, N)

Quá trình mã hóa:

- Thông điệp m được chuyển thành số: m<n
- Bản mã $c = me\ mod\ n$

Quá trình giải mã:

- Bản mã c, c<n
- Bản rõ m = $cd\ mod\ n$

Lưu ý: Khi chọn hai số nguyên tố p và q để sử dụng trong hệ mã hoá RSA, quan trọng nhất là chúng không nên quá gần nhau hoặc quá lệch nhau.

Đánh giá:

Giải thuật RSA là một trong những giải thuật mã hoá công khai (asymmetric cryptography) quan trọng và phổ biến nhất được sử dụng trong lĩnh vực bảo mật thông tin. Dưới đây là một số điểm đánh giá về giải thuật RSA:

- An toàn và Bảo mật: RSA cung cấp một cơ chế mạnh mẽ để bảo vệ thông tin truyền trực tuyến bằng cách sử dụng các khóa công khai và bí mật. Việc phân tích khóa bí mật từ khóa công khai được cho là một bài toán rất khó đối với các thuật toán hiện đại nhưng chưa có bằng chứng toàn diện chứng minh rằng nó không thể được thực hiện.
- Khả năng chống lại các cuộc tấn công: RSA có thể chống lại nhiều loại cuộc tấn công như tấn công brute force (tấn công dò mò), tấn công miền trung gian (man-in-the-middle), và tấn công factorization (phân tích nhân tử). Tuy nhiên, việc sử dụng các cặp khóa ngắn có thể dễ dàng bị tấn công bằng các thuật toán factorization mạnh.

- Hiệu suất: RSA có thể tính toán chậm đối với các khóa dài. Các phép toán như phép nhân số nguyên lớn và tính modulo có thể tốn nhiều thời gian, đặc biệt là với các số nguyên lớn có hàng trăm hoặc hàng nghìn bit.
- Sử dụng rộng rãi: RSA được sử dụng rộng rãi trong các ứng dụng bảo mật như trao đổi khóa, ký số, và mã hoá dữ liệu truyền trực tuyến. Nó được tích hợp vào nhiều giao thức mạng như HTTPS, SSH, và SSL/TLS.
- Quản lý khóa: RSA đòi hỏi quản lý khóa cẩn thận. Quản lý và bảo vệ khóa bí mật là một thách thức đối với việc triển khai và vận hành các hệ thống sử dụng RSA.
- Chi phí tính toán: RSA có thể tốn kém về chi phí tính toán so với các giải thuật mã hoá khác như AES (Advanced Encryption Standard) trong một số tình huống.

Tóm lại, RSA là một giải thuật mã hoá công khai mạnh mẽ và phổ biến, nhưng cũng cần cân nhắc về hiệu suất tính toán và quản lý khóa khi triển khai trong các ứng dụng thực tế.

CHƯƠNG 2. NỘI DUNG THỰC HÀNH

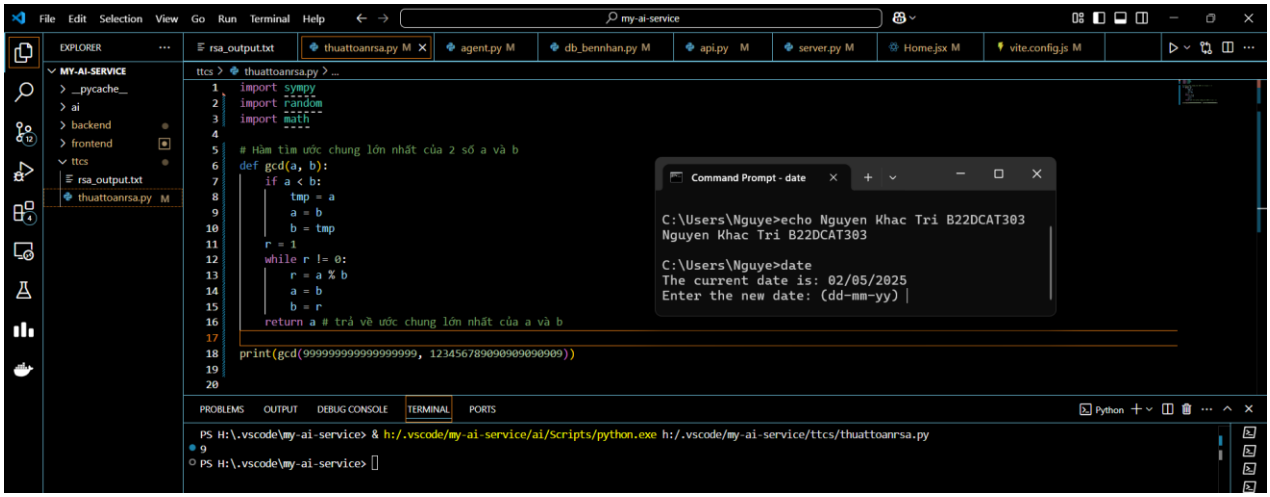
2.1 Chuẩn bị môi trường

- Python 3.11.7

2.2 Các bước thực hiện

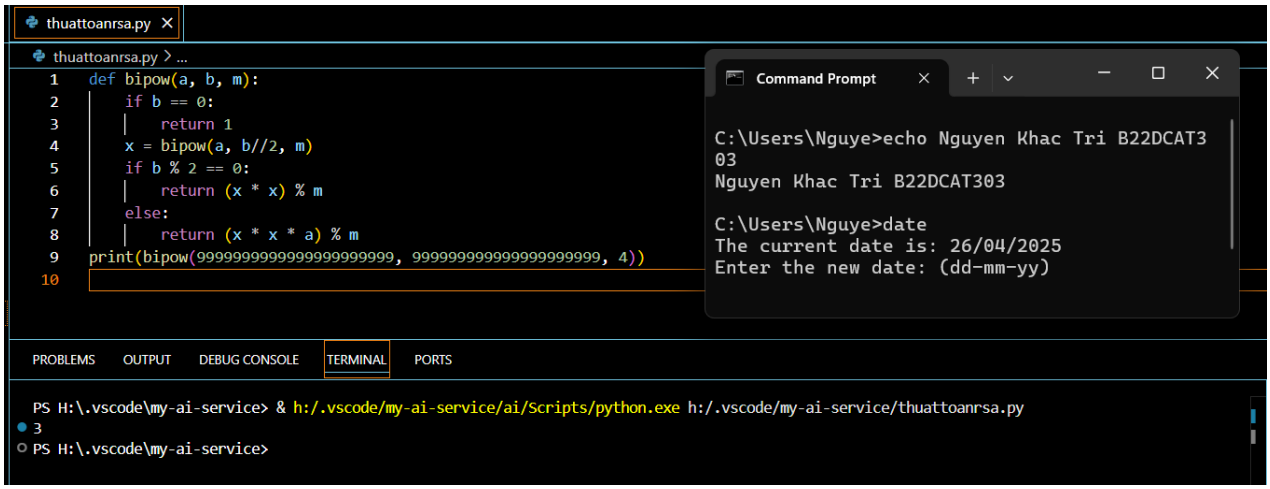
2.2.1 Lập trình thư viện số lớn với các phép toán cơ bản, thử nghiệm chứng minh thư viện hoạt động tốt với các ví dụ phép toán cho số lớn

- Hàm tính ước chung lớn nhất



Hình 1 Hàm tính ước chung lớn nhất

- Hàm tính a^b mod m



Hình 2 Hàm tính a^b mod m

- Hàm tìm số nghịch đảo d sao cho d.e mod m = 1

```python
def multiplicative_inverse(e, phi):
    d = 0
    x1, x2, y1 = 0, 1, 1
    temp_phi = phi

    while e > 0:
        temp1 = temp_phi // e
        temp2 = temp_phi - temp1 * e
        temp_phi = e
        e = temp2

        x = x2 - temp1 * x1
        y = d - temp1 * y1

        x2 = x1
        x1 = x
        d = y1
        y1=y

    if temp_phi == 1:
        return d + phi

print(f'Số d: {multiplicative_inverse(3233, 3120)}')
mod = 3233 * multiplicative_inverse(3233, 3120) % 3120
print(f'Số dư d.e mod phi: {mod}')
```

```
C:\Users\Nguye>echo Nguyen Khac Tri B22DCAT3
03
Nguyen Khac Tri B22DCAT303

C:\Users\Nguye>date
The current date is: 26/04/2025
Enter the new date: (dd-mm-yy)
```

```
PS H:\.vscode\my-ai-service> & h:/.vscode/my-ai-service/ai/Scripts/python.exe h:/.vscode/my-ai-service/thuattoanrsa.py
Số d: 3617
Số dư d.e mod phi: 1
PS H:\.vscode\my-ai-service>
```

Hình 3 Hàm tìm số nghịch đảo d sao cho d.e mod m = 1

- Hàm sinh cặp khóa

```python
def generate_keypair(p, q):
    n = p * q
    phi = (p - 1) * (q - 1)

    # Chọn một số nguyên e sao cho 1 < e <
    e = 65537
    # e = random.randint(1, phi)
    # while math.gcd(e, phi) != 1:
    #     e = random.randint(1, phi)

    # Sử dụng thuật toán Euclid mở rộng để
    d = multiplicative_inverse(e, phi)

    return ((e, n), (d, n))  # (public_key, private_key)
```

```
C:\Users\Nguye>echo Nguyen Khac Tri B22DCAT3
03
Nguyen Khac Tri B22DCAT303

C:\Users\Nguye>date
The current date is: 26/04/2025
Enter the new date: (dd-mm-yy)
```

Hình 4 Hàm sinh khóa, để đơn giản ta chọn e = 65537

- Lập trình giải thuật mã hóa và giải mã

```python
def encrypt(public_key, plaintext):
    e, n = public_key
    # Hàm ord: nhận 1 kí tự làm đầu vào và trả về mã ASCII
    cipher = [pow(ord(char), e, n) for char in plaintext]
    return cipher

def decrypt(private_key, ciphertext):
    d, n = private_key
    # Hàm chr: nhận 1 số nguyên làm đầu vào và trả về kí tự
    plain = [chr(pow(char, d, n)) for char in ciphertext]
    return ''.join(plain)
```

```
C:\Users\Nguye>echo Nguyen Khac Tri B22DCAT3
03
Nguyen Khac Tri B22DCAT303

C:\Users\Nguye>date
The current date is: 26/04/2025
Enter the new date: (dd-mm-yy)
```

Hình 5 Hàm mã hóa và giải mã

2.2.2 Thử nghiệm mã hóa và giải mã chuỗi ký tự: "I am B22DCAT303"

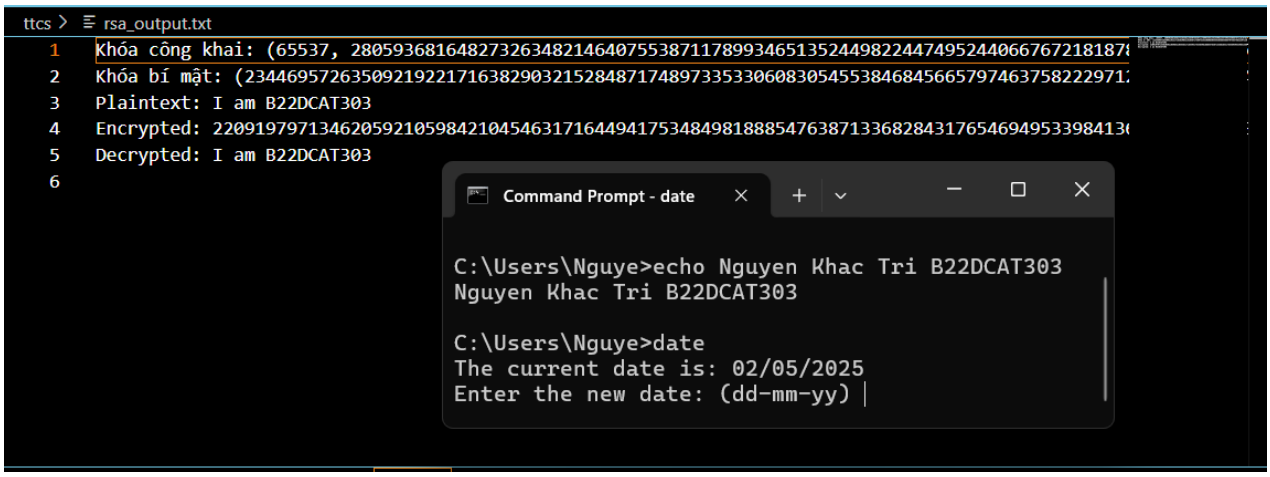
- Thử nghiệm code với mã hóa và giải mã

```python
# Thử nghiệm mã hóa và giải mã
import sympy

# Cách 1: sử dụng giá trị p, q nhỏ
# p = 61
# q = 53

# Cách 2: sinh số nguyên tố lớn ngẫu nhiên (độ dài 1024 bit)
p = sympy.randprime(2**1023, 2**1024)
q = sympy.randprime(2**1023, 2**1024)

# Sinh khóa
public_key, private_key = generate_keypair(p, q)

# Nội dung cần mã hóa
message = "I am B22DCAT303"
print("Plaintext:", message)

# Mã hóa
encrypted_msg = encrypt(public_key, message)
out_encrypted = ' '.join(str(x) for x in encrypted_msg)

# Giải mã
decrypted_msg = decrypt(private_key, encrypted_msg)

# Ghi kết quả vào file
with open("rsa_output.txt", "w", encoding="utf-8") as f:
    f.write(f"Khóa công khai: ({public_key[0]}, {public_key[1]})\n")
    f.write(f"Khóa bí mật: ({private_key[0]}, {private_key[1]})\n")
    f.write(f"Plaintext: {message}\n")

    out_encrypted = ' '.join(str(x) for x in encrypted_msg)
    f.write(f"Encrypted: {out_encrypted}\n")

    f.write(f"Decrypted: {decrypted_msg}\n")
```

```
C:\Users\Nguye>Echo Nguyen Khac Tri B22DCAT303
Nguyen Khac Tri B22DCAT303

C:\Users\Nguye>date
The current date is: 26/04/2025
Enter the new date: (dd-mm-yy)
```

Hình 6 Thử nghiệm code với mã hóa và giải mã

- Kết quả chạy chương trình
  - Mở file lưu kết quả



Hình 7 Kết quả chạy chương trình

TÀI LIỆU THAM KHẢO

[1] Đỗ Xuân Chợ, Bài giảng Mật mã học cơ sở, Học viện Công Nghệ Bưu Chính Viễn Thông, 2021.

[2] Đỗ Xuân Chợ, Bài giảng Mật mã học nâng cao, Học viện Công Nghệ Bưu Chính Viễn Thông, 2021.